



MODEL DEGREE COLLEGE

GONDWANA UNIVERSITY, GADCHIROLI



Index Page

Index number	Title
1	Abstract
2	Introduction
3	Objective
4	Methodology
5	Tools & Technologies
6	Architecture
7	Implementation
8	Results & Outputs
9	Overcoming the Limit
10	Conclusion
11	References

1. Abstract

RAG has emerged as an effective architecture in NLP for enhancing the capabilities of LLMs by integrating external Knowledge Sources. Traditional RAG pipelines rely heavily on vector embeddings and similarity based retrieval mechanisms to identify relevant information. Although effective these approaches introduce several challenges. including the need to segment documents into smaller chunks, potential loss of contextual continuity, and increased system complexity due to embedding generation and vector database management.

This project proposes a Vectorless Reasoning-Based RAG approach that eliminates the dependency on vector embeddings. The system leverages document structure analysis, keyword-based retrieval, and contextual reasoning to identify and navigate relevant sections of text, simulating how humans interact with large documents. By focusing on structural and contextual cues rather than vector similarity, the proposed method preserves contextual integrity while reducing system complexity.

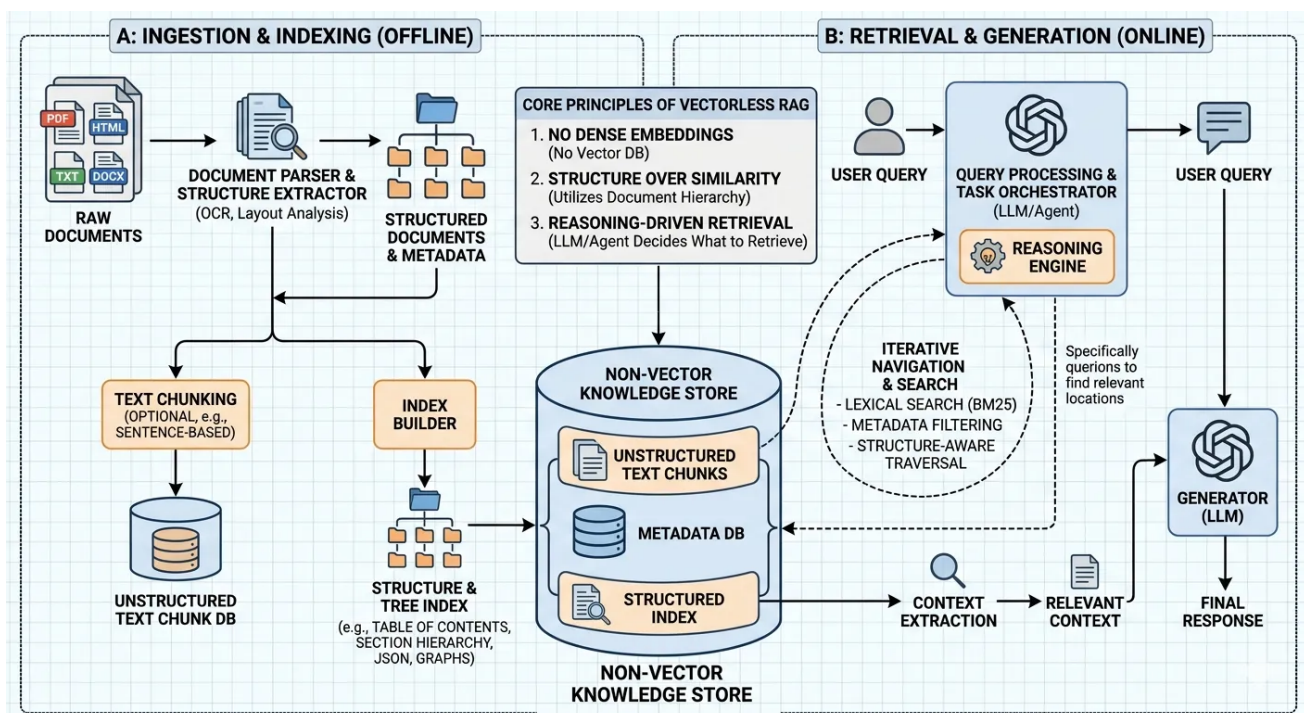
The system is implemented using lightweight NLP techniques and demonstrates that vectorless approaches can serve as a viable alternative for small to medium-scale applications. The results indicate improvements in interpretability, reduced computational overhead, and ease of implementation compared to traditional RAG systems.

2. Introduction

Large Language Models (LLMs) have become powerful engines for document understanding and question answering. However, they are constrained by a fundamental architectural limit: the **context window**, i.e., the maximum number of tokens the model can process at once. Even though recent models support longer contexts, research such as [Chroma's context rot study](#) has shown that model performance deteriorates as context length grows. This makes it challenging for LLMs to accurately interpret and reason over long, complex, domain-specific documents such as financial reports or legal filings.

To mitigate this limitation, Retrieval-Augmented Generation has emerged as an effective solution. Instead of processing entire documents, RAG systems retrieve only the most relevant portions of text based on a user query, thereby optimizing the use of the available context window. However, traditional vector-based RAG approaches rely heavily on static semantic similarity through embeddings, which introduces several limitations. These include the need for document chunking, potential fragmentation of important context, and increased system complexity due to embedding generation and vector database management.

To address these limitations, this project proposes a Vectorless Reasoning-Based RAG approach. Unlike conventional methods, the proposed system constructs a hierarchical representation of documents, enabling the model to navigate content in a structured manner. Inspired by how humans read and analyze documents, the system identifies relevant sections using contextual cues and reasoning, rather than relying solely on semantic similarity. This approach allows the model to dynamically infer which parts of a document are truly relevant, improving contextual understanding while reducing dependency on vector-based infrastructure.



3. Objective

The primary objectives of this project are:

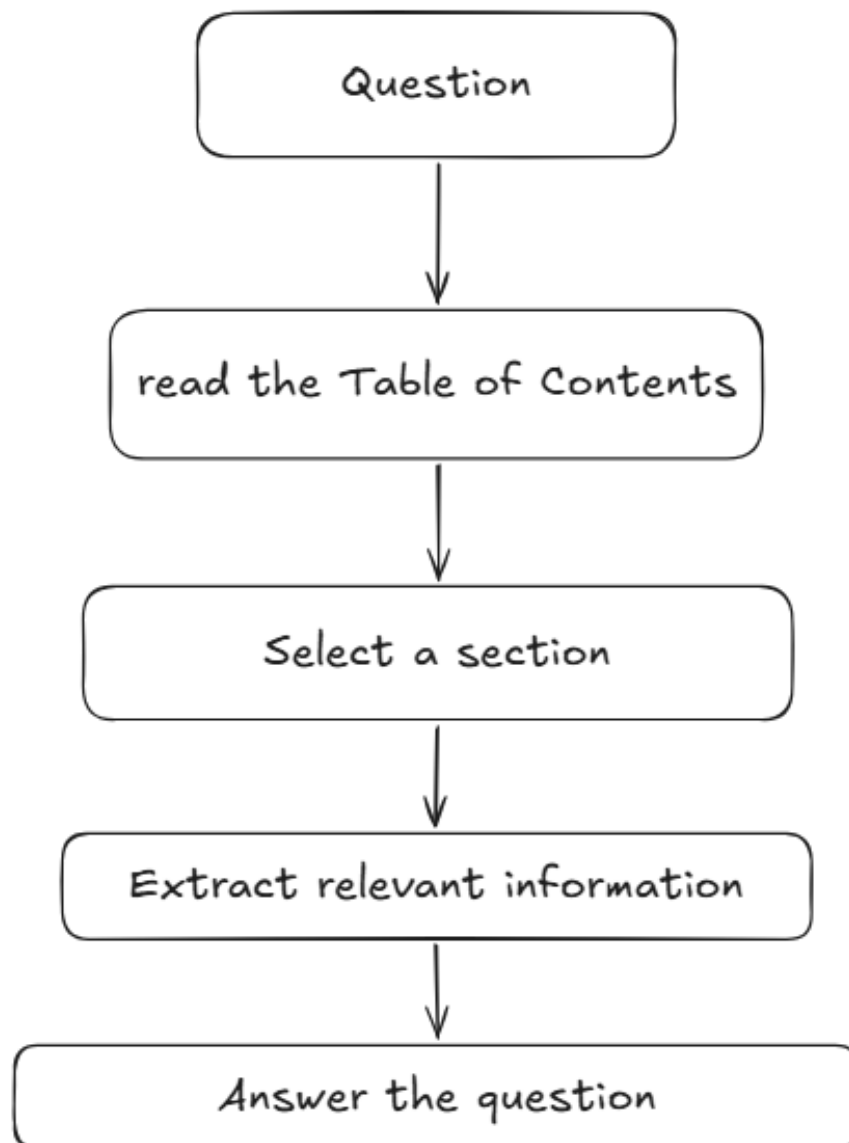
1. To design and implement a vectorless Retrieval-Augmented Generation (RAG) system that eliminates the need for embedding-based retrieval.
2. To develop a reasoning-based document navigation approach that mimics how humans explore and extract information from long documents.
3. To address the limitations of traditional vector-based RAG systems, such as context fragmentation due to chunking and mismatch between semantic similarity and actual relevance.
4. To utilize document structure (e.g., Table of Contents and section hierarchy) for efficient and context-aware information retrieval.
5. To reduce system complexity by avoiding the use of vector databases and embedding generation.
6. To evaluate the effectiveness of the proposed approach in handling long and structured documents.

4. Methodology

When a domain expert needs to find information in a 200-page report, they don't read every page. They:

1. **Scan the table of contents** to understand the document structure
2. **Reason** about which sections are likely to contain the answer
3. **Navigate** to those specific sections
4. **Read** the relevant content
5. **Synthesize** an answer

Instead of vectors, this approach uses structured document navigation.



5. Tools & Technologies

System Requirement

- **Runtime:** Node.js (v18 or higher recommended)
- **Package Manager:** npm / yarn / pnpm
- **Operating System:** Windows, Linux, or macOS
- **Memory:** Minimum 4GB RAM (8GB recommended for local models)

Core Dependencies

- JavaScript (ES2020+)
- Node.js standard libraries (`fs` , `path` , `process` , etc.)

API Providers

Configure one or more of the following providers depending on your use case:

- **OpenAI** – requires API key
 - **Google Gemini** – requires API key
 - **Anthropic (Claude)** – requires API key
- These are used for cloud-based LLM inference.

Local Model Support

- **Ollama** (for running local LLMs)
 - Requires Ollama to be installed and running
 - Supports models like `llama3` , `mistral` , etc.

Network Requirements

Network Requirements

- Internet connection required for:
 - OpenAI
 - Gemini
 - Anthropic
- Not required when using Ollama (local inference)

6. Architecture

Phase 1: Index Construction

The document is transformed into a **hierarchical tree index** — think of it as a smart table of contents with summaries:

```
Document: "Microservices Architecture Guide"
├─ 1. Introduction [pages 1-3]
│   Summary: "Defines microservices, compares with monoliths..."
│   └─ 1.1 What Are Microservices [pages 1-1]
│   └─ 1.2 Why Microservices Matter [pages 1-2]
│   └─ 1.3 Monolith vs Microservices [pages 2-3]
├─ 2. Core Design Principles [pages 3-6]
│   Summary: "Covers single responsibility, loose coupling, API design..."
│   └─ 2.1 Single Responsibility [pages 3-4]
│   └─ 2.2 Loose Coupling [pages 4-5]
│   └─ ...
└─ ..
```

Each node contains:

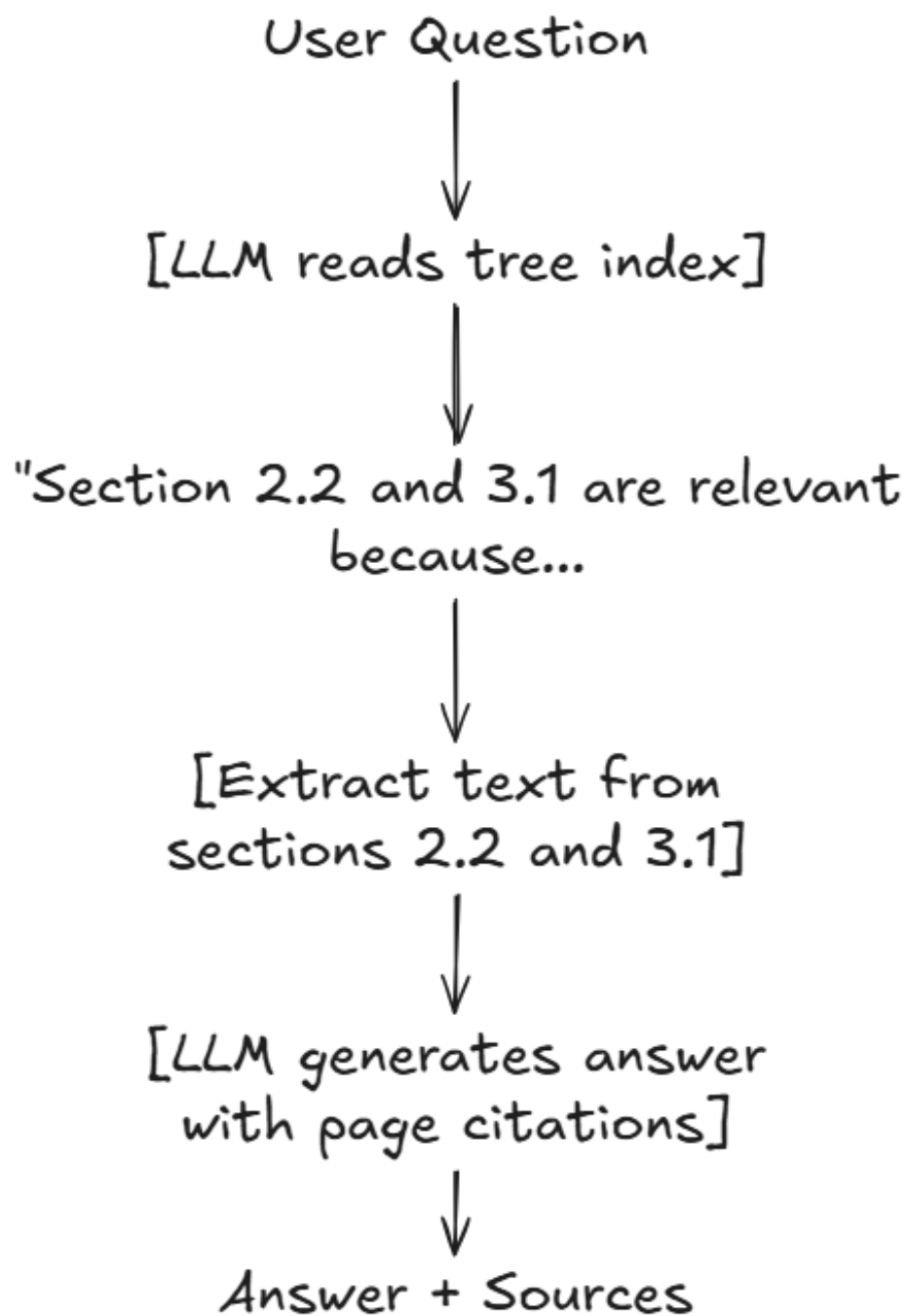
- **Title** — The section heading
- **Page range** — Where this section lives in the document
- **Summary** — A concise description of the section's content
- **Children** — Subsections (creating the hierarchy)

The LLM generates this structure by reading the document and identifying its natural organization.

Phase 2: Reasoning Based Retrieval

When a user asks a question:

1. **Tree Search** — The LLM receives the tree index (titles + summaries, NOT the full text) and reasons about which nodes are relevant
2. **Context Extraction** — The actual text from the selected nodes is extracted
3. **Answer Generation** — The LLM reads the extracted text and generates an answer with citations



7. Implementation

1. indexer.js - Builds the Tree Index

The file does two things

a) Parse the document into "pages"

```
function parseMarkdownIntoPages(filePath, linesPerPage = 40) {
  const content = fs.readFileSync(filePath, "utf-8");
  const lines = content.split("\n");
  const pages = [];

  for (let i = 0; i < lines.length; i += linesPerPage) {
    const pageLines = lines.slice(i, i + linesPerPage);
    pages.push({
      pageNumber: pages.length + 1,
      text: pageLines.join("\n"),
    });
  }
  return pages;
}
```

b) Ask the LLM to build a hierarchical index

The key prompt instructs the LLM to:

1. Read the entire document (tagged with page markers)
2. Identify the natural hierarchy (sections, subsections)
3. Return a JSON tree with title, page range, and summary for each node

The LLM response looks like

```
{
  "title": "Microservices Architecture Guide",
  "start_page": 1,
  "end_page": 8,
  "summary": "A comprehensive guide covering microservices design, communication,
deployment, security, and testing.",
  "children": [
    {
      "title": "Introduction",
      "start_page": 1,
      "end_page": 2,
      "summary": "Defines microservices and compares them with monolithic
architecture.",
      "children": [...]
    }
  ]
}
```

c) LLM API call

Instead of using any SDK, we make raw HTTPS calls. Each provider (OpenAI, Gemini, Claude, Ollama) has its own config for endpoint, auth, request format, and response parsing — all handled by a `PROVIDERS` config object:

```
const PROVIDERS = {
  openai: { hostname: "api.openai.com", path: "/v1/chat/completions", ... },
  gemini: { hostname: "generativelanguage.googleapis.com", ... },
  anthropic: { hostname: "api.anthropic.com", path: "/v1/messages", ... },
  ollama: { hostname: "localhost", port: 11434, ... },
};

function callLLM(apiKey, model, systemPrompt, userPrompt, provider = "openai") {
  const cfg = PROVIDERS[provider];
  // builds request body, headers, and parses response per provider
}
```

2. retriever.js - Reasoning Based Retrieval

a) Tree Search

The LLM receives ONLY the tree structure (titles + summaries, not full text) and reasons about which nodes answer the question:

```
function buildSearchPrompt(query, tree) {
  const slimTree = stripTextFromTree(tree); // remove full text, keep summaries

  return `You are given a user question and a hierarchical tree index...
  QUESTION: ${query}
  DOCUMENT TREE: ${JSON.stringify(slimTree, null, 2)}
  ...select the most specific nodes...`;
}
```

The LLM responds with its reasoning:

```
{
  "thinking": "The question asks about testing strategies. Node 0.6 covers
  Testing Strategies with children for the testing pyramid, E2E testing, and
  performance testing.",
  "selected_nodes": ["0.6.1", "0.6.2", "0.6.3"]
}
```

b) Context Extraction

We look up the selected nodes in the tree, find their page ranges, and extract the actual text

```
function extractContext(selectedNodeIds, tree, pages) {
  // Find each node, get its page range, collect text
  for (const nodeId of selectedNodeIds) {
    const node = allNodes.find((n) => n.node_id === nodeId);
    const nodePages = pages.filter(
      (p) => p.pageNumber >= node.start_page && p.pageNumber <= node.end_page
    );
    // ...
  }
}
```

c) Answer Generation

The extracted text is sent to the LLM with instructions to answer the question and cite sources:

```
{
  "answer": "The microservices testing strategy follows a pyramid model with
three levels...",
  "sources": [
    {"node_id": "0.6.1", "title": "The Testing Pyramid", "pages": "6-7"}
  ]
}
```

3. index.js - CLI & Interactive Loop

Ties everything together:

Flags

Usage:

```
node index.js --file <path>           Build index and start querying
node index.js --index <json> --file <path> Load existing index
```

Options:

```
--file, -f      Path to markdown/text file (required)
--index, -i     Path to saved index JSON (skip indexing step)
--provider, -p  LLM provider: openai, gemini, anthropic, ollama (default:
openai)
--model, -m     Model name (default: auto per provider)
--help, -h      Show this help
```

Environment (set in .env or shell):

```
OPENAI_API_KEY   For OpenAI (default)
GEMINI_API_KEY   For Google Gemini
ANTHROPIC_API_KEY For Anthropic Claude
(Ollama needs no key – runs locally)
```

1. Parses CLI arguments
2. Builds or loads the index
3. Enters an interactive query loop where you type questions and get cited answers

8. Results & Outputs

Step 1: Build the Hierarchical tree index

```
bluecouch@Couch MINGW64 /d/code/learnCode/ai30days/vectorless/hoopa
$ node index.js --file ./sample-data/sample-doc.md --provider ollama
Provider: ollama | Model: llama3
Ollama: "llama3" not installed; using "llama3:latest"

Parsing document: D:\code\learnCode\ai30days\vectorless\hoopa\sample-data\sample-doc.md
Found 6 pages

Building hierarchical tree index via LLM (ollama/llama3:latest)...

Document Index:
0 Microservices Architecture [pages 1-6]
  -> A comprehensive guide to microservices architecture, covering its principles, benefits, and challenges.
  0.1 Introduction [pages 1-2]
    -> An overview of microservices architecture and its importance in modern software development.
  0.2 Principles [pages 3-4]
    -> The core principles of microservices architecture, including single responsibility, loose coupling, independent deployment, and data ownership.
    0.2.1 Single Responsibility [pages 3-3]
      -> Each microservice should have a single responsibility and be designed to perform one specific task.
    0.2.2 Loose Coupling [pages 4-4]
      -> Microservices should be loosely coupled, allowing for independent development, deployment, and scaling.
  0.3 Benefits [pages 5-6]
    -> The benefits of microservices architecture, including scalability, flexibility, and improved maintainability.
    0.3.1 Scalability [pages 5-5]
      -> Microservices allow for independent scaling of individual services, enabling more efficient use of resources.
    0.3.2 Flexibility [pages 6-6]
      -> Microservices enable the use of different programming languages, frameworks, and databases for each service.

Index saved to: D:\code\learnCode\ai30days\vectorless\hoopa\sample-data\sample-doc_index.json
```

Step 2: Ask Questions

```
Vectorless RAG - Interactive Query Mode
Type your question and press Enter.
Type "quit" or "exit" to stop.
Type "tree" to view the document index.

? Your question: What are the main communication patterns between microservices?

Query: "What are the main communication patterns between microservices?"

? Your question: What are the main communication patterns between microservices?

Query: "What are the main communication patterns between microservices?"

Step 1: Searching tree index (LLM reasoning)...
Thinking: The question is asking about the main communication patterns between microservices, so we need to focus on nodes that discuss how services interact with each other. The 'Principles' section (0.2) seems relevant as it covers loose coupling and data ownership, which are key aspects of service communication. Within this section, the 'Loose Coupling' node (0.2.2) is particularly important as it highlights the importance of independence in microservices.
Selected nodes: 0.2, 0.2.2

Step 2: Extracting context from selected nodes...
Extracted 2 sections (Principles, Loose Coupling)

Step 3: Generating answer from context...

ANSWER:
Event-driven architectures and service meshes are two main communication patterns between microservices.

Sources:
  - Principles (Pages 3-4)

? Your question: 
```

You can always skip the indexing step on subsequent runs:

```
node index.js --index ./sample-data/sample-doc_index.json --file ./sample-data/sample-doc.md
```

Using your own documents

Point `--file` at any markdown or text file:

```
node index.js --file ~/my-report.md
node index.js --file ~/meeting-notes.txt
```

Switching Providers

```
node index.js --file ./sample-data/sample-doc.md --provider gemini

node index.js --file ./sample-data/sample-doc.md --provider anthropic

node index.js --file ./sample-data/sample-doc.md --provider ollama --model llama3
```

9. Overcoming The Limitations

1. Query–Knowledge Space Mismatch

Instead of relying solely on embedding similarity search, the LLM **uses reasoning to infer which section is likely to contain the answer**. It can “think” about document structure: e.g., “*Debt trends are usually in the financial summary section or Appendix G — let’s look there.*” This dynamic inference bridges the gap between *query meaning* and *information location*.

2. Semantic Similarity ≠ True Relevance

Reasoning-based retrieval emphasizes **contextual relevance**, not just similarity. The model reads the Table of Contents (ToC) or PageIndex structure, interprets the query’s intent, and **navigates** to sections that actually contain the answer, even if their language is different. This mirrors how humans find answers: by *understanding* the question, not just matching words.

3. Hard Chunking Breaks Semantic Integrity

Rather than chunking arbitrarily, reasoning-based RAG retrieves **semantically coherent sections** (e.g., full pages, sections, or chapters). If the model detects that a section is incomplete, it **iteratively fetches neighboring sections** (e.g., next page or sub-node) until context is sufficient. This preserves logical continuity and minimizes hallucination.

4. Inability to Integrate Chat History

Retrieval is **context-aware**: the model uses prior conversation history to refine its understanding of the current question. For instance, if the user previously asked about “financial assets,” and now asks, “What about liabilities?”, the retriever knows to explore *the same report section* under liabilities. This enables coherent, multi-step exploration across multiple turns.

5. Poor Handling of In-Document References

By leveraging the **hierarchical structure**, reasoning-based retrieval can *follow references like a human reader*. When it encounters a phrase like “*see Appendix G*”, the LLM navigates the index tree to that section and retrieves the relevant data. This allows accurate cross-referencing without manual link-building.

Limitation	Vector-based RAG	Reasoning-based RAG
1. Query–Knowledge Mismatch	Matches surface-level similarity; often misses true context	Uses inference to identify the most relevant document sections
2. Similarity ≠ Relevance	Retrieves semantically similar but irrelevant chunks	Retrieves contextually relevant information
3. Hard Chunking	Fixed-length chunks fragment meaning	Retrieves coherent sections dynamically
4. No Chat Context	Each query is isolated	Multi-turn reasoning considers prior context
5. Cross-References	Fails to follow internal document links	Follows in-text references via ToC/PageIndex reasoning

10. Conclusion

Vector-based RAG searches for similar text whereas reasoning-based RAG *thinks* about *where* to look and *why*. By combining structured document representations (like ToC Trees) with iterative reasoning, reasoning-based RAG enables LLMs to retrieve the **relevant information**, not just **similar information**, paving the way for a new generation of intelligent document understanding systems.

11. References

1. <https://techcommunity.microsoft.com/blog/azuredevcommunityblog/vectorless-reasoning-based-rag-a-new-approach-to-retrieval-augmented-generation/4502238>
2. <https://pageindex.ai/blog/pageindex-intro>
3. <https://rlancemartin.github.io/2025/04/03/vibe-code/>
4. <https://www.trychroma.com/research/context-rot>